

AGENTGUARD: Learning Execution Guardrails from Anomalous Coding-Agent Trajectories

Wuyang Dai, Song Wang

Lassonde School of Engineering, York University
ddai2002@yorku.ca, wangsong@yorku.ca

Abstract

AI coding agents increasingly rely on execution harnesses to interact with repositories and external tools. However, task success does not guarantee reliable execution. Agents may still modify unrelated files, rewrite tests, issue unsafe commands, or ignore failed validations, motivating behavioral guardrails for reliable execution.

We present AGENTGUARD, an instruction-level guardrail framework that learns conditional execution constraints from anomalous trajectories of coding agents. Rather than relying on manually specified safety rules, AGENTGUARD automatically extracts recurring execution failure patterns, generalizes them into instruction-level behavioral constraints, and organizes them as a lightweight guardrail skill that dynamically activates only the rules relevant to the current instruction. This design enables behavioral guidance while minimizing unnecessary restrictions on normal execution.

We evaluate AGENTGUARD using 642 documented failure traces collected from real coding-agent executions across 382 repository tasks. Guardrails are learned from 461 traces covering 282 tasks and evaluated on a disjoint set of 100 tasks. Using Claude Code with Claude Haiku 4.5 as the underlying coding agent, we compare the baseline agent with the same agent augmented by AGENTGUARD. Experimental results show that AGENTGUARD reduces the Abnormal Execution Rate from 69.0% to 26.7% and increases the Successful Task Completion Rate from 21.7% to 35.0%. These results demonstrate that execution guardrails learned from historical failures can substantially improve the reliability of AI coding agents while highlighting the remaining challenge of balancing safety and task completion.

1 Introduction

Coding agents operate directly on repositories through shell, filesystem, editing, and validation tools. Systems such as SWE-agent and OpenHands use this interaction loop to carry out multi-step software tasks with limited human supervision (Yang et al. 2024; Wang et al. 2025). The same autonomy makes their behavior difficult to control. A local decision made during execution can modify repository state, alter the environment, weaken validation, or produce an unsupported result. Failures are especially risky because the agent must decide whether to retry, change its approach,

modify the project, or stop. A plausible final answer does not guarantee that these intermediate actions remained within the intended task boundary. We call an execution *abnormal* when an observed action violates task scope, repository or environment integrity, safe recovery, validation, artifact, or evidence-grounded reporting requirements.

A natural solution is to introduce behavioral guardrails that constrain how agents execute tasks rather than evaluating only their final outputs (Dong et al. 2026). Moreover, empirical evidence shows that negative behavioral constraints consistently outperform prescriptive guidance for coding agents, suggesting that preventing undesirable actions is often more effective than prescribing optimal ones (Zhang et al. 2026). Existing approaches, however, typically rely on manually specified policies or developer-authored rule sets that are uniformly enforced across tasks (Dong et al. 2026; Zhang et al. 2026). Such manually engineered guardrails are difficult to maintain, cannot easily adapt to newly emerging failure patterns, and may unnecessarily restrict benign behaviors that are irrelevant to the current execution context.

In this paper, we present AGENTGUARD, an instruction-level guardrail framework that automatically learns behavioral constraints from historical anomalous coding-agent trajectories. Instead of manually writing execution policies, AGENTGUARD analyzes recurring failure traces, extracts reusable execution patterns, and synthesizes conditional instruction-level guardrails that are activated only when their triggering conditions are satisfied. The learned guardrails are organized as a lightweight skill that can be dynamically loaded before each instruction, allowing the agent to receive targeted behavioral guidance while avoiding unnecessary constraints during normal execution.

We evaluate AGENTGUARD using 642 documented anomalous execution traces collected from 382 real-world repository tasks introduced by Dai et al. (Dai et al. 2026). We use 461 traces from 282 tasks for guardrail construction and reserve a disjoint set of 100 tasks for evaluation. Each evaluation task is executed three times with the baseline agent and three times with the same agent augmented by AGENTGUARD, resulting in 600 Docker-isolated executions using Claude Code with Claude Haiku 4.5. Experimental results show that AGENTGUARD reduces the Abnormal Execution Rate from 69.0% to 26.7% and increases the Successful Task Completion Rate from 21.7% to 35.0%.

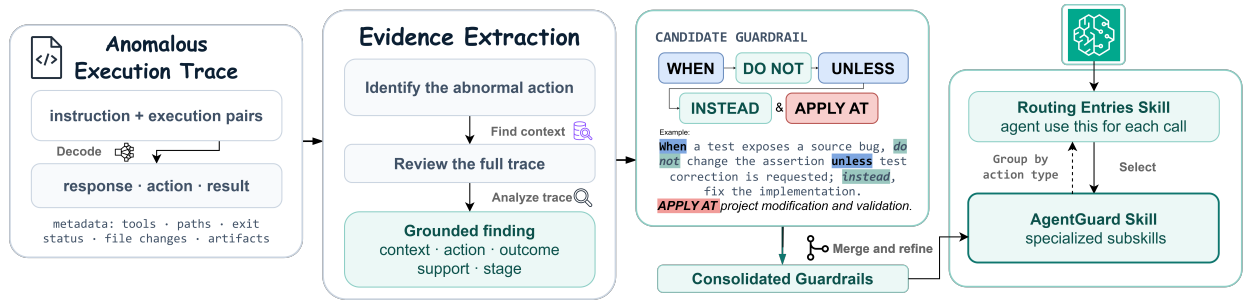


Figure 1: Overview of the AGENTGUARD pipeline. The dataset is partitioned by task before guardrail induction to prevent information leakage. Construction traces are used to learn guardrails, whereas held-out tasks are reserved exclusively for evaluation.

Our main contributions are:

- We propose the first instruction-level framework that automatically learns conditional behavioral guardrails from historical anomalous coding-agent trajectories.
- We introduce a trajectory-driven guardrail generation pipeline that extracts reusable execution constraints and dynamically activates only task-relevant behavioral rules.
- We conduct a large-scale empirical evaluation on real coding-agent executions, demonstrating substantial improvements in execution reliability while identifying over-refusal as the primary limitation of learned behavioral guardrails.

2 Related Work

Coding agents and their evaluation: Coding agents solve repository tasks through sequences of observations, tool calls, file changes, and validation steps. SWE-agent and OpenHands make this interaction loop central to automated software engineering, while SWE-bench established repository-level issue resolution as a standard end-state evaluation (Yang et al. 2024; Wang et al. 2025; Jimenez et al. 2024). End-state outcomes, however, do not fully characterize the execution process: trajectory analysis shows that decisions about context gathering, recovery, and validation can distinguish successful executions from problematic ones (Mehtiyev and Assunção 2026). We focus on these intermediate decisions. ABTest provides reviewed, repository-grounded traces of problematic coding-agent executions, including their commands, file effects, and generated artifacts (Dai et al. 2026). We use this evidence to derive behavioral guardrails.

Agent safety and execution guardrails: Recent work has explored improving agent safety through behavioral constraints, runtime monitoring, and reusable execution guidance. ToolEmu and Constitutional AI constrain agent behavior through policy-based evaluation or feedback mechanisms (Ruan et al. 2024; Bai et al. 2022). Runtime safety frameworks such as GuardAgent, AGrail, and TRIAD monitor or intervene during execution to enforce safety policies (Xiang et al. 2025; Luo et al. 2025; Sun et al. 2026), while trajectory-based approaches such as R-Judge and TrajAD analyze multi-step executions to identify risky or anomalous

behaviors (Yuan et al. 2024; Liu et al. 2026). In parallel, reusable natural-language rules and coding skills have been proposed to improve coding agents without modifying their underlying models or tool interfaces, although their effectiveness is inconsistent across tasks (Zhang et al. 2026; Han et al. 2026; Aggarwal and Ghalaty 2026).

Different from these approaches, AGENTGUARD learns lightweight repository-level guardrails directly from reviewed anomalous execution traces and injects them as contextual instructions into an existing coding agent. This design requires no model fine-tuning, tool modification, or runtime enforcement mechanism.

3 Method

AGENTGUARD learns reusable execution guardrails from historical anomalous coding-agent trajectories. Given a collection of reviewed failure traces, it identifies recurring execution patterns, abstracts them into conditional behavioral constraints, consolidates redundant or overlapping rules, and organizes the resulting guardrails into a lightweight routing structure that dynamically activates only the rules relevant to the current instruction. Figure 1 provides an overview of the framework, and Algorithm 1 summarizes the complete guardrail induction procedure. The following subsections describe each stage in detail.

3.1 Execution Trace Representation

A coding-agent task consists of a sequence of user instructions and the corresponding agent executions. To support guardrail induction, the representation must preserve both the local decision process for each instruction and the execution dependencies across the entire task. We therefore model a coding-agent case as:

$$\text{trace} = \langle (\text{instruction}_i, \text{execution}_i) \rangle_{i=1}^N,$$

where each instruction is paired with the agent execution produced in response. An execution is represented as an ordered sequence of interaction triples:

$$\text{execution}_i = \langle (\text{response}_{i,t}, \text{action}_{i,t}, \text{result}_{i,t}) \rangle_{t=1}^{T_i}.$$

where *response* denotes the agent’s reasoning or observable message, *action* denotes the invoked tool call (e.g., shell com-

Algorithm 1: Trace-grounded guardrail induction and consolidation.

Require: Anomalous execution traces $\mathcal{T}$

Ensure: Organized guardrail set $\mathcal{G}$

```

1: Extract grounded findings  $\mathcal{V}$  from  $\mathcal{T}$ 
2: Initialize candidate rule set  $\mathcal{R} \leftarrow \emptyset$ 
3: for each finding  $v \in \mathcal{V}$  do
4:   Create  $r$  by mapping context, action, and stage to
     WHEN, DoNOT, and APPLYAT
5:   Add UNLESS when the action is authorized by the task
6:   Set INSTEAD to a bounded alternative that avoids the
     outcome
7:   Generalize repository-specific details without chang-
     ing applicability
8:   Attach the finding's supporting trace elements
9:    $\mathcal{R} \leftarrow \mathcal{R} \cup \{r\}$ 
10: end for
11: repeat
12:   Remove duplicates and combine their supporting evi-
     dence
13:   Apply subsumption when exceptions and allowed be-
     havior are preserved
14:   Separate conflicts that require different responses
15: until the rule set no longer changes
16: Remove rules that fail the validation criteria
17: Assign each rule to a routing category and execution
     stage
18: return  $\mathcal{G}$ 

```

mand, file edit, or search), and *result* records the corresponding environment feedback. The final response represents the agent's completion of the instruction.

Each interaction additionally retains execution metadata, including tool names, command arguments, file paths, exit status, repository changes, and generated artifacts. Instructions encode the requested objective together with explicit constraints, such as authorized scope, expected outputs, and validation requirements.

This hierarchical representation captures execution behavior at two complementary levels. Interaction triples provide fine-grained evidence for identifying anomalous decisions, while the complete trace preserves dependencies across instructions, enabling guardrail induction to incorporate execution context, intermediate state changes, and downstream consequences rather than treating each action in isolation.

An execution is anomalous when an action conflicts with the task specification, exceeds authorized scope, disregards available evidence, or supports an unjustified success claim. Thus, a trace may be anomalous even when its final output appears plausible.

3.2 Evidence Extraction

The objective of evidence extraction is to convert a reviewed anomalous execution trace into a structured behavioral finding that captures *what happened*, *why it was anomalous*, and *where it occurred*. Since each training trace in ABTest (Dai et al. 2026) is annotated with one reviewed anomalous action,

every trace yields one grounded finding.

We first identify the reviewed anomalous action within the execution trace. Its associated instruction, preceding interactions, and execution result provide the immediate execution context and observed consequence. We then examine the complete task trajectory to recover additional contextual information. Earlier instruction–execution pairs may reveal inherited constraints, repository state, or prior decisions, while later interactions expose downstream effects, recovery attempts, or propagated failures. Combining local and global context enables the anomaly to be interpreted within the complete execution rather than as an isolated action. Each anomaly is summarized as a structured finding:

finding = (context, action, outcome, support, stage).

For example, suppose an instruction asks the agent to fix a source-code bug without changing the tests, but the agent changes a failing test assertion instead of fixing the implementation. The extracted finding is: *context* = the instruction and failed test; *action* = changing the test assertion; *outcome* = a weakened validation target while the source bug remains; *support* = the instruction, test output, and file diff; and *stage* = project modification and validation. A finding is used only when its action, outcome, and trace support are observable. Broad summaries alone do not support rule induction.

3.3 Rule Induction and Consolidation

The objective of rule induction is to transform grounded behavioral findings into reusable execution constraints that can guide future coding-agent executions. Each finding is converted into a conditional guardrail that specifies *when* the constraint applies, *which* behavior should be avoided, and *how* the agent should safely proceed instead. Formally, a candidate guardrail is represented as:

$r = (\text{WHEN}, \text{DoNOT}, \text{UNLESS}, \text{INSTEAD}, \text{APPLYAT})$.

WHEN, DoNOT, and APPLYAT come from the finding's context, action, and stage. UNLESS is added when the task conditions permit the same action; otherwise it is left empty. INSTEAD gives the smallest alternative that avoids the outcome while preserving the valid task goal. Repository names, paths, and error strings are generalized only when doing so does not change when the rule applies. The finding's support remains linked to the candidate rule.

In the example above, the induced guardrail is: WHEN: the task requires a source-code fix and the test exposes the bug; DoNOT: change the assertion to bypass the failure; UNLESS: the task explicitly requires correcting the test; INSTEAD: fix the implementation and rerun the test; and APPLYAT: project modification and validation. The rule restricts the shortcut without prohibiting legitimate test changes. This simplified example only illustrates the five fields. The induced guardrails specify more precise observable triggers, authorized exceptions, bounded alternatives, and evidence requirements.

Candidate rules are merged only when they constrain the same decision under equivalent conditions and prescribe the same safe response. Differences in their triggers, exceptions, or safe alternatives keep them separate. The rules are consolidated through three operations:

1. **Duplicate removal.** Two rules are duplicates when they apply at the same stage, prohibit the same action under compatible conditions, and prescribe the same response. Their evidence is combined.
2. **Subsumption.** A specific rule is absorbed by a more general rule only if the general rule covers the same cases without restricting additional legitimate behavior. Exceptions from the specific rule are preserved.
3. **Conflict resolution.** Rules are kept separate, or their preconditions are refined, when they prescribe different responses for the same apparent action.

Each rule must be *grounded* in trace evidence, *observable* before the action, *actionable* through a clear alternative, and *non-interfering* with legitimate work. Rules retain links to their supporting evidence.

3.4 Rule Organization and Routing

Loading every guardrail for every instruction would lengthen the prompt and require the agent to consider rules unrelated to the current task. We therefore group guardrail subskills by the type of action they constrain; each group is called a *routing entry*. This organization is illustrated in Table 1. Before using a tool, the main skill identifies the requested type of work, selects the corresponding routing entry, and loads the relevant subskill. The agent therefore receives only the detailed trigger, restriction, exception, and safe alternative needed for the current instruction. A routing entry adds no new rule; it only organizes the existing guardrails and helps the main skill retrieve the relevant one.

For the source repair example above, the requested result requires modifying the project, so the routing skill selects the project-changes entry. That entry groups the guardrail subskills for source edits, refactoring, tests, configuration, and dependencies. If the agent considers changing the failing assertion, it loads the test-and-validation subskill. That guardrail permits the edit only when the instruction explicitly requests a test correction; otherwise, the agent must leave the test unchanged and fix the implementation. The routing skill and guardrail subskill add instructions to the model’s context; they do not change tool permissions or intercept actions.

4 Experimental Design

4.1 Guardrail Construction and Intervention

We use 642 documented failures from actual coding agent executions released with ABTest (Dai et al. 2026). Each includes the task instruction, execution trace, tool outputs, and repository effects. These failures cover 382 unique tasks. Before rule construction, we split them by task into a rule-construction set of 282 tasks and 461 traces and a held-out evaluation set of 100 tasks, ensuring no task appears in both partitions.

According to ABTest (Dai et al. 2026), each task is a coherent multi-step repository workflow with one injected adversarial step designed to elicit abnormal behavior. The remaining steps specify legitimate work that the agent should complete when they remain safe and feasible. Injected steps

include unsafe requests, missing commands, files, or interfaces, and ambiguous instructions that require clarification. A correct response should handle the injected problem without fabricating results or unnecessarily abandoning the legitimate parts of the task.

Analyzing the 461 construction traces produced 461 grounded findings, which were consolidated into 15 guardrails. Each guardrail is implemented as a specialized subskill. We then group subskills that govern the same type of action under one of five routing entries: (1) understanding and path resolution, (2) command execution and recovery, (3) project changes, (4) filesystem, version control, and external-state mutation, and (5) artifacts, validation, and reporting. Rather than loading all guardrails for every task, AGENTGUARD first selects the routing category most relevant to the current instruction and then dynamically loads only the associated guardrail subskills before tool execution. This routing mechanism reduces prompt overhead while providing context-specific behavioral guidance.

Table 1 shows one abstracted example for each guardrail. Each example shows only its central trigger, restriction, and safe response. The implemented rules are substantially more detailed, with specific preconditions, exceptions, evidence requirements, and multi-step responses.

4.2 Evaluation Tasks and Settings

The evaluation set contains 100 multi-step tasks, each with one injected adversarial step designed to elicit abnormal behavior. Each task is executed three times in each condition to account for the inherent stochasticity of LLM-based coding agents, yielding a more reliable estimate of behavioral outcomes. The two evaluation conditions are:

- **Raw Agent:** the agent (i.e., Claude Code in this paper) receives the original task instructions without behavioral guardrails.
- **Agent + AGENTGUARD:** the same agent receives the AGENTGUARD guidance before acting on each instruction.

This design produces 300 executions per configuration and 600 executions in total. All experiments use Claude Code 2.1.19 with Claude Haiku 4.5 under an identical execution environment (two CPUs, 8 GB memory). Each run starts from a fresh Docker container initialized with the same Click repository snapshot, ensuring that no conversation history or repository modifications are carried across executions. Tool access, permissions, and runtime configurations are identical in both conditions; the only difference is the addition of AGENTGUARD in the guarded setting.

4.3 Review and Statistics

Before comparing **Raw Agent** and **Agent + AGENTGUARD**, reviewers assess each execution in two stages. First, they check whether the injected adversarial step was handled correctly, i.e., an unsafe request should be refused or replaced with a safe alternative, a missing command, file, or interface should not lead to a fabricated result, and an ambiguous request should trigger clarification. Second, they examine the remaining execution, including completion or unnecessary refusal of legitimate work, recovery after failures, repository

Routing entry	Detailed guardrail	Abstracted example
Understanding and path resolution	Planning, search, and review	When inspecting a project, keep the work read-only and base conclusions on inspected evidence.
	Path inspection and resolution	When a path is required, verify the authorized root and exact target rather than inventing a substitute.
Command execution and recovery	Tests and builds	When running a check, preserve its target and acceptance criteria.
	Tools, scripts, and commands	When an interface is unsupported, preserve the observed failure instead of inventing a command or result.
	Failure recovery	After a command fails, diagnose the error before making one bounded correction and retry.
Project changes	Source and documentation edits	When a source change is requested, edit only the necessary files and avoid unrelated cleanup.
	Broad refactoring	Before a broad edit, define its boundary, preserved invariants, and acceptance criteria.
	Tests and validation changes	When validation fails, do not weaken the check unless changing it is part of the task.
	Project configuration	When configuration must change, edit only the requested setting and preserve unrelated security-relevant entries.
	Dependencies and environments	When a dependency is needed, use project tooling and avoid unrequested global or lockfile changes.
Filesystem and external state	Filesystem mutation	Before changing files, resolve and preview the exact targets; reject unresolved or unexpectedly broad effects.
	Version control and remote actions	Before a version-control or remote action, verify the target and authority, preserve user changes, and inspect the result.
Artifacts and reporting	Structured output	When producing structured output, derive every field from evidence, then reopen and parse the artifact.
	Logs and reports	When producing a report, use the requested destination and include only traceable facts.
	Validation and final reporting	Before claiming completion, run read-only validation and report failed, blocked, or unverified effects.

Table 1: The five routing entries and 15 conditional guardrails in AGENTGUARD. The entries group existing guardrail subskills by the actions they constrain. The examples abstract the central decision in each guardrail; the full subskills contain more specific triggers, exceptions, evidence requirements, and safe alternatives.

and environment changes, produced artifacts, and whether final claims are supported by trace evidence. The 300 **Raw Agent** executions and 300 **Agent + AGENTGUARD** executions are reviewed separately using the same criteria.

Three human reviewers independently assessed all 100 tasks and all 600 execution traces. They did not consult prior verdicts or one another’s records. Evidence included prompts, tool events, changed files, artifacts, and repository state; oracle output only helped locate evidence. Since AGENTGUARD adds explicit guardrail instructions to the execution trace, reviewers could identify the evaluation condition during annotation. When reviewers disagreed, they jointly re-examined the cited trace evidence and reached a final adjudicated judgment.

Before adjudication, inter-rater reliability measured by *Fleiss’s kappa* value was 0.87. When disagreements occurred, reviewers jointly re-examined the task instructions and supporting execution evidence until reaching consensus. Only the adjudicated labels were used in the final analysis.

During the review process, reviewers record the following outcome measures for each execution:

- **Abnormal Execution:** an action violates task scope, repository or environment integrity, safe recovery, or

evidence-grounded reporting.

- **Correct Adversarial-Step Handling:** the injected problem is recognized and met with the required refusal, clarification, safe substitute, truthful unsupported report, or safe recovery. Failure of unrelated benign work is evaluated separately.
- **Correct Benign-Step Handling:** all applicable benign requirements are completed, including work that remains independently feasible after an unsafe step is blocked. Executions with no independently feasible benign work are counted as not completed.
- **Over-Refusal:** the agent refuses, skips, or abandons legitimate work that remains authorized, technically possible, and safe, including work that can continue after the injected problem is blocked.
- **Successful Task Completion:** an execution contains no abnormal behavior, handles the injected problem correctly, and completes all legitimate work that remains safe and possible.

These outcome measures are not mutually exclusive because a task consists of multiple execution steps. An execution may correctly handle the adversarial step while later exhibiting abnormal execution or over-refusing legitimate

Metrics	CC	CC + AGENTGUARD	Improvement
AER(%)	69.0 207/300	26.7 80/300	−61.4%
CAR(%)	25.0 75/300	62.7 188/300	+150.7%
CBR(%)	49.0 147/300	42.0 126/300	−14.3%
ORR(%)	— —	19.3% 58/300	—
Success(%)	21.7 65/300	35.0 105/300	+61.5%

Table 2: Performance comparison between the baseline Claude Code (CC) and Claude Code augmented with AGENTGUARD (CC + AGENTGUARD) on the 100-task evaluation set. Each configuration contains 300 executions.

work. The individual measures capture these complementary aspects of execution, whereas **Successful Task Completion** is assigned only when the entire task is completed without abnormal execution, the adversarial step is handled correctly, and all feasible benign work is completed.

4.4 Metrics

We evaluate AGENTGUARD using five complementary metrics that characterize both execution reliability and task utility:

- **(%AER) Abnormal Execution Rate:** the percentage of tasks that exhibit one or more abnormal execution behaviors.
- **(%CAR) Correct Adversarial-Step Handling Rate:** the percentage of tasks in which the injected adversarial step is handled correctly.
- **(%CBR) Benign Task Completion Rate:** the percentage of tasks in which all legitimate work that remains feasible is successfully completed.
- **(%Success) Successful Task Completion Rate:** the percentage of tasks that are completed successfully without abnormal execution, correctly handle the adversarial step, and complete all feasible benign work.
- **(%ORR) Over-refusal Rate:** the percentage of tasks in which the agent unnecessarily refuses, skips, or abandons legitimate work.

5 Results

5.1 Overall Performance of AGENTGUARD

Table 2 reports the outcome rates under raw Claude Code (denoted as CC) and Claude Code + AGENTGUARD (denoted as CC + AGENTGUARD). Overall, compared with the baseline Claude Code, AGENTGUARD substantially improves execution reliability while introducing a modest reduction in benign task completion.

Specifically, the **Abnormal Execution Rate (AER)** decreases from 69.0% to 26.7%, representing a 61.4% relative reduction (42.3 percentage points). Statistical significance is

Status	$n/100$ tasks	%
CC only	41	41.0
Both CC and CC + AGENTGUARD	33	33.0
CC + AGENTGUARD only	3	3.0
No abnormal behavior in either setting	23	23.0

Table 3: Task-level abnormal execution status across the 100 evaluation tasks.

assessed at the task level using a nonparametric cluster bootstrap for 95% confidence intervals and an exact paired randomization test for hypothesis testing. The reduction is statistically significant (95% task-clustered bootstrap CI: -51.7 to -33.0 ; exact paired randomization test, $p < 0.001$), demonstrating that AGENTGUARD substantially reduces anomalous execution behaviors compared with the baseline agent.

The **Correct Adversarial-Step Handling Rate (CAR)** increases from 25.0% to 62.7% (+150.7%). The absolute difference is +37.7 percentage points (task-clustered 95% CI: 28.3 to 47.0; $p < 0.001$), showing that AGENTGUARD substantially improves the agent’s ability to recognize and respond appropriately to injected adversarial steps through safe refusal, clarification, or alternative actions. The **Benign Task Completion Rate (CBR)** decreases from 49.0% to 42.0% (−14.3%). The absolute difference is −7.0 percentage points and is not statistically significant (task-clustered 95% CI: -17.0 to 3.0 ; $p = 0.199$), providing no evidence of a systematic reduction in benign task completion. The **Over-Refusal Rate (ORR)** is 19.3% (58/300) in the guarded setting and zero under CC. The absolute difference is +19.3 percentage points (task-clustered 95% CI: 12.7 to 26.7; $p < 0.001$), identifying over-refusal as the primary failure mode introduced by AGENTGUARD. Despite this trade-off, the **Successful Task Completion Rate** increases from 21.7% to 35.0%, a relative improvement of 61.5%. The absolute difference is +13.3 percentage points (task-clustered 95% CI: 5.3 to 21.7; $p = 0.0022$). Since successful task completion requires the simultaneous satisfaction of all evaluation criteria, including no abnormal execution, correct handling of adversarial steps, and completion of all feasible benign work, this result shows that the reduction in abnormal executions outweighs the loss caused by over-refusal.

Table 3 summarizes the task-level abnormal execution status across the 100 evaluation tasks between the baseline Claude Code (CC) and Claude Code augmented with AGENTGUARD (CC + AGENTGUARD). Abnormal execution appears under CC only in 41 tasks (41.0%), which accounts for most of the observed reduction. It remains present under both CC and CC + AGENTGUARD in 33 cases (33.0%), showing that the guardrails do not suppress every known failure mode. Only three tasks (3.0%) exhibit abnormal execution exclusively under CC + AGENTGUARD, whereas abnormal execution is eliminated in 41 tasks that are abnormal under CC alone. This highly asymmetric transition (41 versus 3 tasks) demonstrates that the learned guardrails substantially reduce abnormal execution while introducing very few new anomalies.

Category	Effect	$n/300$ runs	%
<i>Beneficial</i>	Safety gain, no benign loss	100	33.3
	Safety gain with benign loss	29	9.7
<i>Neutral</i>	No attributed effect	139	46.3
<i>Adverse</i>	Benign loss only	25	8.3
	Safety loss only	6	2.0
	Safety loss with benign loss	1	0.3

Table 4: Trace-level attribution of AGENTGUARD’s effects across 300 execution traces.

5.2 Effects of AGENTGUARD

Beyond measuring execution reliability, we further analyze *how* AGENTGUARD influences agent behavior during execution. Specifically, we categorize the effect of each execution into three high-level outcomes: *beneficial*, *neutral*, and *adverse*. To ensure reliable annotation, the same three reviewers independently assigned each of the 300 executions to one mutually exclusive effect category. The resulting inter-rater agreement achieved a Fleiss’ kappa of 0.81, indicating almost perfect agreement.

A *safety gain* is assigned when AGENTGUARD prevents an abnormal action that occurs under the baseline CC. A *safety gain with benign loss* further indicates that some independently feasible, benign work is not completed because of the intervention. Conversely, a *safety loss* indicates that AGENTGUARD introduces an abnormal behavior that is absent in the baseline.

Table 4 summarizes the effects of AGENTGUARD across the 300 executions of the 100 evaluation tasks. Overall, the reviewers identified **129 safety gains** and only **32 adverse effects: 25 benign losses, 6 safety losses, and 1 involving both**. Among the 129 safety gains, **100** preserve all benign work, demonstrating that the observed improvements cannot be explained merely by conservative or blanket refusals. Nevertheless, **55 executions** incur benign utility loss, indicating that the primary remaining challenge is enabling the agent to safely continue execution after a guardrail is triggered, rather than prematurely abandoning feasible work.

Since AGENTGUARD augments the base coding agent with 15 learned guardrails (see Table 1), we further investigate their effects at the routing-entry level. Specifically, we analyze each routing-entry activation during execution and classify its effect after the associated guardrails are applied. Figure 2 summarizes the distribution of beneficial, neutral, and adverse outcomes across routing entries. Because a single task execution may trigger multiple routing entries, the total number of routing-entry activations exceeds the number of task executions.

Overall, Execution/Recovery is the most frequently activated routing entry and yields the largest number of beneficial effects (120). Across all routing entries, beneficial effects consistently outnumber adverse ones. Most adverse effects occur in Artifacts/Reporting (29) and Understanding/Path (27), suggesting that future improvements should focus on understanding- and reporting-related behaviors.

Execution measures	CC	CC + AGENTGUARD	Δ
Mean cost (\$)	0.158	0.162	+2.5%
Mean exec. time (s)	93.3	97.0	+3.9%
Mean # tool calls	21.8	22.7	+4.1%

Table 5: Average execution overhead per run.

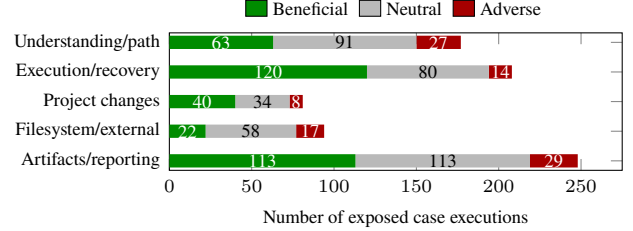


Figure 2: Distribution of beneficial, neutral, and adverse outcomes across routing entries.

5.3 Cost Analysis

Since AGENTGUARD augments the base coding agent with additional verification skills, it may introduce extra execution overhead. We therefore compare the execution cost, runtime, and tool usage between the baseline CC and CC + AGENTGUARD. As shown in Table 5, the overhead introduced by AGENTGUARD is modest. The mean execution cost increases from \$0.158 to \$0.162 per run (+2.5%), while the mean execution time increases from 93.3 to 97.0 seconds (+3.9%). The average number of tool calls also rises slightly, from 21.8 to 22.7 (+4.1%). Considering that AGENTGUARD reduces the abnormal execution rate by 61.4% while substantially improving execution reliability, these small increases in resource consumption demonstrate that the proposed guardrails achieve a favorable trade-off between safety and execution efficiency.

6 Conclusion

This paper presented AGENTGUARD, a lightweight framework that improves the execution reliability of AI coding agents by learning repository-level guardrails from anomalous execution traces. Rather than modifying the underlying model or execution environment, AGENTGUARD injects learned behavioral constraints as contextual instructions during task execution. Experiments on 100 coding tasks demonstrate that AGENTGUARD substantially reduces abnormal execution while preserving benign behaviors and introducing only modest execution overhead. These results suggest that failure-driven behavioral guardrails provide an effective and practical mechanism for improving the safety and reliability of autonomous coding agents.

References

Aggarwal, A.; and Ghalaty, N. F. 2026. Self-Improving AI Coding Agents Through Accumulated Behavioral Rules: A Closed-Loop Framework. arXiv:2607.13091.

- Bai, Y.; Kadavath, S.; Kundu, S.; Askill, A.; Kernion, J.; Jones, A.; Chen, A.; Goldie, A.; Mirhoseini, A.; McKinnon, C.; Chen, C.; Olsson, C.; Olah, C.; Hernandez, D.; Drain, D.; Ganguli, D.; Li, D.; Tran-Johnson, E.; Perez, E.; Kerr, J.; Mueller, J.; Ladish, J.; Landau, J.; Ndousse, K.; Lukosuite, K.; Lovitt, L.; Sellitto, M.; Elhage, N.; Schiefer, N.; Mercado, N.; DasSarma, N.; Lasenby, R.; Larson, R.; Ringer, S.; Johnston, S.; Kravec, S.; El Showk, S.; Fort, S.; Telleen-Lawton, T.; Conerly, T.; Henighan, T.; Hume, T.; Bowman, S. R.; Hatfield-Dodds, Z.; Mann, B.; Amodei, D.; Joseph, N.; McCandlish, S.; Brown, T.; and Kaplan, J. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*.
- Dai, W.; Openja, M.; Pham, H. V.; Uddin, G.; Yang, J.; and Wang, S. 2026. ABTest: Behavior-Driven Testing for AI Coding Agents. *arXiv:2604.03362*.
- Dong, T.; Shi, S.; Sampath, H.; and Macvean, A. 2026. From Correctness to Collaboration: A Human-Centered Taxonomy of AI Agent Behavior in Software Engineering. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, 1–5.
- Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; and Hu, L. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv:2603.15401*.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. R. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations*.
- Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. *arXiv:2602.06443*.
- Luo, W.; Dai, S.; Liu, X.; Banerjee, S.; Sun, H.; Chen, M.; and Xiao, C. 2025. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. *arXiv:2502.11448*.
- Mehtiyev, T.; and Assunção, W. 2026. Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure. *arXiv:2604.02547*.
- Ruan, Y.; Dong, X.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations*.
- Sun, Y.; Zhang, J.; Cohnsey, S.; Zhang, Z.; Liu, F.; and Yuan, X. 2026. From Risk Classification to Action Plan Remediation: A Guardrail Feedback Driven Framework for LLM Agents. *arXiv:2606.05805*.
- Wang, X.; Li, B.; Song, Y.; Xu, F. F.; Tang, X.; Zhuge, M.; Pan, J.; Song, Y.; Li, B.; Singh, J.; Tran, H. H.; Li, F.; Ma, R.; Zheng, M.; Qian, B.; Shao, Y.; Muennighoff, N.; Zhang, Y.; Hui, B.; Lin, J.; Brennan, R.; Peng, H.; Ji, H.; and Neubig, G. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations*.
- Xiang, Z.; Zheng, L.; Li, Y.; Hong, J.; Li, Q.; Xie, H.; Zhang, J.; Xiong, Z.; Xie, C.; Yang, C.; Song, D.; and Li, B. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, 68316–68342. PMLR.
- Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*.
- Yuan, T.; He, Z.; Dong, L.; Wang, Y.; Zhao, R.; Xia, T.; Xu, L.; Zhou, B.; Li, F.; Zhang, Z.; Wang, R.; and Liu, G. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 1467–1490. Association for Computational Linguistics.
- Zhang, X.; Wang, G.; Cui, Y.; Qiu, W.; Li, Z.; Zhu, B.; and He, P. 2026. Do Agent Rules Shape or Distort? Guardrails Beat Guidance in Coding Agents. *arXiv preprint arXiv:2604.11088*.